Mapping the Workflow

Immersion Week SF · Day 1 · Session 1.2

55

Minutes

You leave with a diagram the team shares

What This Session Covers

01 Decomposing the problem into steps

02 Turning the map into an Artifact

03 Naming what each step needs

04 Finding how the riskiest step fails

From input to output: how?

From 1.1 to Now

Problem

One precise sentence

vs

Workflow

The path to solve it

How We Get There

01

Break

Split it into steps

02

Draw

Make it an Artifact

03

Annotate

Name what each needs

04

Stress

Find how it breaks

01

Decompose Into Steps

Workflow

Ordered steps from raw input to delivered output.

Step

One change, with a named input and output.

Every Step Has Three Parts

Input

What arrives, and its shape

Transform

The one thing this step does

Output

What leaves, ready for next

How to Decompose

01 Name the output

What the user finally gets

02 Work backwards

What must exist before it

03 Stop at raw input

What you actually receive

Start From the Output

Forward drifts toward what is easy

Backwards keeps every step earning

You spot the missing middle sooner

Vague Step, Concrete Step

Vague	Concrete
Process the data	Parse CSV to records
Analyze it	Score each row 0 to 1
Use AI	Claude drafts a summary
Show results	Render a ranked table

The Granularity Test

One person could own it

You could test it alone

It fits in a morning

Too Coarse, Too Fine

Too coarse

Build the pipeline

Hides the hard part

Cannot be assigned

Too fine

Open the file

Twenty trivial boxes

Noise, not a map

Prompt to Decompose

Our problem:
"<paste the 1.1 statement>"

Work BACKWARDS from the output.
List the steps, each with its
input and output.
Stop at the raw input we have.
5-9 steps. No tools yet.

A Worked Example

01

Intake

Resume files arrive

02

Parse

Text out of each file

03

Score

Match against the role

04

Report

Ranked list with gaps

Common Decomposition Mistakes

Starting after the messy part

One step doing three things

No step for getting the data

The output nobody defined

Linear or Branching

Linear

One path start to end

Easy to demo

Pick this for Day 2

Branching

Cases split the path

Harder to finish

Note it, defer it

The happy path

The one route where nothing fails.

Mark the Happy Path

One line through the map

Every branch labelled later

Edge cases listed, not built

"A gap in the map is a gap in Day 2."

- Why the middle matters

02

Make It an Artifact

Claude Artifact

Standalone content Claude builds, shared by link.

Described or Drawn

Described	Drawn
Lives in a transcript	One picture, one meaning
Each person pictures it	Disagreement shows fast
Re-explained each time	Sent as a link

What an Artifact Gives You

See

The whole path at once

Revise

Change it in seconds

Share

One link, whole team

Build the Diagram

01 Paste the steps

Your list from section one

02 Ask for a diagram

Boxes and arrows, top down

03 Fix it out loud

Revise until it matches

Prompt for the Diagram

Here are our workflow steps:
<paste the list>

Build an Artifact: a flow diagram, top to bottom.
One box per step.
Label every arrow with what passes between the steps.

What the Diagram Must Show

Raw input, where it enters

One box per step

What passes on each arrow

The delivered output

Diagram Conventions

Top to bottom reads fastest

One verb per box

Keep labels under five words

Revise It in Front of Everyone

Ask for the change out loud

Re-read the new version together

Stop when nobody objects

Artifact Pitfalls

A pretty diagram nobody rechecks

Boxes with no arrow labels

The link never actually shared

Where the Map Lives

Chat	vs	Project
Scrolls away by noon		Beside the problem

03

Name What Each Step Needs

Four Kinds of Need

Need	What it means
Data	Something must be supplied
Tool	Code, API, or model call
Routine	It runs on a schedule
Human	A person decides

Annotate Every Box

One need per step, minimum

Say which of the four it is

Unknowns get marked, not skipped

Claude Routine

A prompt Claude runs on a schedule, unprompted.

When a Step Is a Routine

It repeats on a clock

Nobody should have to remember

Late is worse than imperfect

A Step as a Routine

Every weekday at 8:00

1. Read new files in /intake
2. Score each against the role
3. Append to the ranked sheet
4. Flag anything scoring low

Human decision

A step where a person judges and the system waits.

Keep a Human When

Being wrong is expensive

The call needs context you lack

Someone must be accountable

Step to Need

Step	What it needs
Intake files	Data plus a routine
Parse text	A tool
Score match	A model call
Approve list	A human decision

The riskiest step

The one whose failure wastes the most time.

Find It

01 Mark unknowns

Boxes you cannot describe

02 Ask who blocks

Which stalls everything after

03 Pick one

Only one gets this attention

Prompt for Failure Modes

Our riskiest step:
"<describe the step>"

Give the THREE most likely ways
this fails in practice.
Most likely first.
For each: the early warning sign.
No fixes yet.

Where Steps Usually Fail

Input

Arrives wrong or not at all

Transform

Works, but not on real data

Access

The key, quota, or approval

What to Do With Three Failures

Build the warning sign in first

Stub the step if it may not land

Give the risky box its best owner

Which Step Gets the Attention

	Intake	Parse	Score
Unknown	**Yes**	Yes	Yes
Blocks rest	No	Yes	No
Hard to fix	Yes	No	No

Write the Risk Down

Put it on the diagram

Name the warning sign

Say who is watching for it

Done With This Session When

The map runs input to output

It is an Artifact with a link

Every box names what it needs

The riskiest step has three failures

What You Leave With

1

Shared diagram

4

Kinds of need

3

Named failures

Next: Dividing the Work

Give every step one owner

Run workstreams in parallel

Put the board in GitHub